

TP 11 — La station météo

Semaine 11 ■ Fichiers dans NSI/TP11/ ■ 2 h

Objectifs

- Implémenter et adapter les algorithmes de référence
- Mesurer expérimentalement le coût linéaire
- Produire un bilan lisible à partir de données brutes

Partie A — Les capteurs — `meteo.py`

(50 min)

On dispose des relevés de température d'un mois (un par jour) :

```
temperatures = [12.1, 14.0, 9.5, 11.2, 15.8, 17.3, 16.0,
                 13.4, 10.1, 8.7, 7.9, 11.5, 14.8, 18.2,
                 19.5, 21.0, 20.3, 17.6, 15.2, 12.8, 10.4,
                 9.1, 11.7, 13.9, 16.4, 18.8, 20.1, 19.2,
                 16.7, 14.3]
```

Écrire sans `max`, `min`, `sum` :

1. `maximum(t)`, `minimum(t)`, `moyenne(t)` — les classiques.
2. `jour_le_plus_chaud(t)` — le **numéro du jour** (indice + 1) du maximum.
3. `nb_jours_gel(t)` — nombre de jours de gel (température négative); tester avec un tableau contenant des négatifs.
4. `amplitude_max(t)` — le plus grand écart de température entre deux jours **consécutifs** (comparer `t[i]` et `t[i + 1]`).
5. `serie_chaude(t, seuil)` — longueur de la plus longue série de jours consécutifs au-dessus du seuil ★★ (deux compteurs : la série en cours, le record).

Puis un bilan f-strings :

```
Mois : max 21.0 (jour 16), min 7.9, moyenne 14.5
Plus longue vague > 15 degrés : 5 jours
```

Partie B — Mesurer le coût linéaire — `cout.py`

(40 min)

1. Écrire `contient(t, x)` (recherche séquentielle du cours).
2. Générer des tableaux aléatoires de tailles 10 000, 100 000, 1 000 000 :

```
import random
t = [random.randint(0, 10**9) for _ in range(taille)]
```

3. Chronométrer la recherche d'une valeur **absente** (`x = -1`) :

```
import time
debut = time.perf_counter()
contient(t, -1)
duree = time.perf_counter() - debut
```

4. Noter les trois durées dans un tableau (commentaire). Quand la taille est multipliée par 10, la durée est multipliée par environ combien ? Conclure sur le coût.

5. ★ Refaire avec $x = t[0]$ (présent au début). Expliquer l'écart spectaculaire.

Partie C — Défi ★★ — la seconde plus grande

(20 min)

`deux_plus_grands(t)` — renvoie le couple (plus grand, deuxième plus grand) en **un seul parcours**. Attention au piège du tableau `[7, 7, 3]` : décider (et documenter) si « deuxième » accepte les doublons.